

Dokumentacja końcowa

Projekt: Wizualizacja grafu planarnego

Wykonały: Piasecka Milena, Kułaga Ewa

1. Wprowadzenie

Celem projektu jest stworzenie aplikacji, której zadaniem jest wyznaczanie współrzędnych wierzchołków grafu planarnego w sposób umożliwiający jego estetyczną wizualizację. Aplikacja składa się z dwóch części: części C, która odpowiada za implementację algorytmu Fruchterman-Reingolda, a część w Javie odpowiada za graficzną prezentację grafu.

2. Zakres funkcjonalny systemu

Aplikacja w części C pozwala na sterowanie programem za pomocą argumentów linii poleceń. Program odczytuje graf zapisany w pliku, tworzy strukturę danych, a następnie generuje współrzędne wierzchołków. Wyniki, w zależności od wyboru użytkownika zapisywane są w pliku tekstowym lub binarnym. Część w Javie jest sterowana za pomocą interfejsu. Po wciśnięciu przycisków *wczytaj graf* i *wczytaj współrzędne*, a następnie wybranie odpowiednich plików na ekranie wyświetli się wizualizacja danego grafu

3. Sposób uruchomienia programu

Program uruchamiany jest z poziomu linii poleceń. Ogólna składnia wywołania:

```
./graf <plik_wejściowy> [opcje]
```

Dostępne opcje: -b – zapis do pliku binarnego, -t – zapis do pliku tekstowego. W przypadku niepodania parametru program nie zadziała. Graficzną prezentację grafu można bezpośrednio uruchomić z poziomu terminala. W tym celu należy skompilować plik komendą 'javac <nazwa pliku>', a następnie go uruchomić: 'java <nazwa pliku>'.

Innym sposobem jest użycie gotowego środowiska programistycznego, który ma wbudowany kompilator. W tym przypadku proces kompilacji i uruchomienia odbywa się automatycznie. Użytkownik musi tylko nacisnąć zieloną strzałkę.

4. Format danych wejściowych

Dane wejściowe należy zapisać w pliku tekstowym w podanym formacie:

<nazwa_krawędzi> <wierzchołek_A> <wierzchołek_B> <waga>

Przykład:

AB 1 2 1

BC 2 3 1

CD 3 4 1

DB 4 2 1.407

Wierzchołki muszą być liczbami całkowitymi. Waga musi być większa od zera.

5. Format danych wyjściowych

Wynik może być zapisany w dwóch postaciach o podanych formatach:

Format tekstowy:

<wierzchołek> <x> <y>

Format binarny:

- liczba wierzchołków

- tablica struktur (id, x, y)

6. Struktura implementacyjna systemu

Część w C:

Cały program składa się z 4 plików pomocniczych wraz z plikami nagłówkowymi, pliku main oraz pliku Makefile.

Dane grafu przechowywane są w strukturze (graph.h) zawierającej listę wierzchołków oraz listę krawędzi, co umożliwia łatwe przetwarzanie danych w kolejnych etapach projektu.

```
#ifndef GRAPH_H
// Wierzchołek
typedef struct {
    int id;           // identyfikator wierzchołka
    double x;         // współrzędna X
    double y;         // współrzędna Y
} Vertex;

// Krawędź
typedef struct {
    char name[32];    // nazwa krawędzi
    int u;             // początek
    int v;             // koniec
    double weight;     // waga
} Edge;

// Graf
typedef struct {
    Vertex* vertices; // tablica wierzchołków
    Edge* edges;       // tablica krawędzi
    int n_vertices;    // liczba wierzchołków
    int n_edges;       // liczba krawędzi
} Graph;

#endif
```

Moduł parsowania wczytuje dane z pliku i tworzy strukturę grafu. Składa się on z plików pars.h i pars.c. W pliku pars.h znajduje się struktura nowego elementu oraz deklaracje funkcji przypisania nowego elementu i czytania pliku.

```
#ifndef PARS_H
#define PARS_H

#include "graph.h"

typedef struct Element {
    Edge dane;
    struct Element *next;
} Element;

Element* dodaj_element(Element *glowa, const char *name, int u, int v, double weight);
Element* czytaj_plik(int argc, char *argv[]);

#endif
```

Funkcja dodaj_element za pomocą malloca alokuje pamięć dla nowego elementu w dynamicznej tablicy, pola struktury wypełnia przekazanymi wartościami, a następnie ustawia go na początek listy.

```
Element* dodaj_element(Element *glowa, const char *name, int u, int v, double weight) {
    Element *nowy = malloc(sizeof(Element));

    if(nowy == NULL)
        return glowa;

    strncpy(nowy->dane.name, name, 31);
    nowy->dane.name[31] = '\0';
    nowy->dane.u = u;
    nowy->dane.v = v;
    nowy->dane.weight = weight;

    nowy->next=glowa;

    return nowy;
}
```

Funkcja `czytaj_plik` przyjmuje plik z danymi grafu. Jeżeli plik nie zostanie podany w linii poleceń program nie zadziała. Funkcja czyta plik i dopóki otrzymuje odpowiednie parametry zapisuje dane do dynamicznej tablicy, używając poprzednio zaimplementowanej funkcji. Po zakończeniu odczytu plik jest zamykany, a funkcja zwraca wskaźnik na pierwszy element stworzonej listy.

```
Element * czytaj_plik(int argc, char *argv[]){

    if(argc < 2) {
        printf("Nie podano nazwy pliku");
        return NULL;
    }

    FILE *f = fopen(argv[1], "r");

    if(f == NULL)
        return NULL;

    Element *glowa = NULL;

    int u,v;
    char name[32];
    double weight;

    while(fscanf(f,"%31s %d %d %lf", name, &u, &v, &weight ) == 4) {
        if (u < 0 || v < 0 || weight <= 0) {
            printf("Wprowadzone wartosci sa bledne\n");
            break;
        }

        glowa = dodaj_element(glowa, name, u,v,weight);
    }

    fclose(f);
    return glowa;
}
```

W celu wyznaczenia współrzędnych dla wizualizacji grafu został użyty algorytm Fruchterman–Reingold. Algorytm opiera się na analogii fizycznej, gdzie wierzchołki grafu traktuje się jako ładunki, które odpychają się nawzajem co zapobiega nakładaniu się węzłów na siebie. Krawędzie łączące wierzchołki zachowują się jak sprężyny, które przyciągają ze sobą punkty, co zapobiega zbyt nadmiernemu rozproszeniu się grafu. Podstawa obliczeń jest wyznaczenie odległości pomiędzy wierzchołkami zależnej od obszaru. Liczy się ją ze wzoru:

$$k = C \sqrt{\frac{AREA}{n}}$$

Gdzie:

C – stała, przyjęta jako 1

AREA – powierzchnia obszaru, przyjęta jako 10 000

n – liczba wierzchołków

Siły oddziaływań obliczane są ze wzorów:

$$\text{Siła odpychania: } f(d) = \frac{k^2}{d}$$

$$\text{Siła przyciągania: } f(d) = \frac{d^2}{k}$$

```
//siły odpychające
for (int v = 0; v < n; v++) {
    for (int u = v + 1; u < n; u++) {

        double dxv = g->vertices[v].x - g->vertices[u].x;
        double dyv = g->vertices[v].y - g->vertices[u].y;

        double dist = sqrt(dxv * dxv + dyv * dyv) + EPSILON;

        double force = (k * k) / dist;

        dx[v] += (dxv / dist) * force;
        dy[v] += (dyv / dist) * force;

        dx[u] -= (dxv / dist) * force;
        dy[u] -= (dyv / dist) * force;
    }
}
```

```

// sily przyciągające – krawędzie
for (int e = 0; e < g->n_edges; e++) {

    int u_id = g->edges[e].u;
    int v_id = g->edges[e].v;

    int u = -1;
    int v = -1;

    // znajdź indeksy wierzchołków
    for (int i = 0; i < n; i++) {
        if (g->vertices[i].id == u_id)
            u = i;
        if (g->vertices[i].id == v_id)
            v = i;
    }

    if (u == -1 || v == -1)
        continue;

    double dxv = g->vertices[v].x - g->vertices[u].x;
    double dyv = g->vertices[v].y - g->vertices[u].y;

    double dist = sqrt(dxv * dxv + dyv * dyv) + EPSILON;

    double force = (dist * dist) / k;

    dx[v] -= (dxv / dist) * force;
    dy[v] -= (dyv / dist) * force;

    dx[u] += (dxv / dist) * force;
    dy[u] += (dyv / dist) * force;
}

```

Plik graph_utilis służy jako funkcja pomocnicza dla wykonywania głównego algorytmu. Zamienia ona listę na strukturę graph, aby algorytm miał szybki dostęp do danych.

Funkcja najpierw przechodzi po liście w celu zliczenia krawędzi, a następnie zapisuje je w tablicy.

```
// policz krawędzie
Element* temp = list;
while (temp != NULL) {
    g->n_edges++;
    temp = temp->next;
}

g->edges = malloc(g->n_edges * sizeof(Edge));
if (g->edges == NULL) {
    free(g);
    return NULL;
}

// kopiuje krawędzie
temp = list;
int i = 0;
while (temp != NULL) {
    g->edges[i++] = temp->dane;
    temp = temp->next;
}
```


Pomocnicza funkcja pozwala na znalezienie powtarzających się numerów węzłów, a następnie utworzenie im osobnych obiektów typu Vertex.

```
// znajdź wierzchołki
int max = g->n_edges * 2;
g->vertices = malloc(max * sizeof(Vertex));
if (g->vertices == NULL) {
    free(g->edges);
    free(g);
    return NULL;
}

for (int i = 0; i < g->n_edges; i++) {
    int u = g->edges[i].u;
    int v = g->edges[i].v;

    int found_u = 0, found_v = 0;

    for (int j = 0; j < g->n_vertices; j++) {
        if (g->vertices[j].id == u)
            found_u = 1;
        if (g->vertices[j].id == v)
            found_v = 1;
    }

    if (found_u == 0) {
        g->vertices[g->n_vertices].id = u;
        g->vertices[g->n_vertices].x = 0.0;
        g->vertices[g->n_vertices].y = 0.0;
        g->n_vertices++;
    }

    if (found_v == 0) {
        g->vertices[g->n_vertices].id = v;
        g->vertices[g->n_vertices].x = 0.0;
        g->vertices[g->n_vertices].y = 0.0;
        g->n_vertices++;
    }
}
```

Moduł wyjścia odpowiada za zapisu danych do pliku tekstowego lub binarnego.

Plik output.h to interfejs modułu wyjścia, w którym deklarujemy funkcje i umożliwia nam korzystanie ze struktur:

```
#ifndef OUTPUT_H
#define OUTPUT_H

#include "graph.h"

// Zapis do pliku tekstowego
int save_output_text(const char* filename, Graph* g);

// Zapis do pliku binarnego
int save_output_binary(const char* filename, Graph* g);

#endif
```

W pliku output.c inicjalizujemy wszystkie funkcje podane w output.h.

Zapis danych realizowany jest w dwóch formatach: tekstowym oraz binarnym. Format tekstowy umożliwia łatwą analizę wyników przez użytkownika, natomiast format binarny zapewnia większą wydajność i mniejszy rozmiar pliku.

```
//
int save_output_text(const char* filename, Graph* g) {
    if (g == NULL)
        return 0;

    FILE* f = fopen(filename, "w");
    if (f == NULL) {
        printf("Error: cannot open file %s\n", filename);
        return 0;
    }

    for (int i = 0; i < g->n_vertices; i++) {
        fprintf(f, "%d %.3f %.3f\n",
                g->vertices[i].id,
                g->vertices[i].x,
                g->vertices[i].y);
    }

    fclose(f);
    return 1;
}
```

```
//
int save_output_binary(const char* filename, Graph* g) {
    if (g == NULL)
        return 0;

    FILE* f = fopen(filename, "wb");
    if (f == NULL) {
        printf("Error: cannot open file %s\n", filename);
        return 0;
    }

    // zapis liczby wierzchołków
    fwrite(&g->n_vertices, sizeof(int), 1, f);

    // zapis tablicy Vertex
    fwrite(g->vertices, sizeof(Vertex), g->n_vertices, f);

    fclose(f);
    return 1;
}
```

Aby usprawnić kompilację został użyty plik Makefile, który kompiluje wszystkie pliki oraz czyści pliki, które powstały w trakcie kompilacji

```
CC = cc
CFLAGS = -Wall -Wextra

graf: fruchterman.o graph_utils.o main.o output.o pars.o
    $(CC) fruchterman.o graph_utils.o main.o output.o pars.o -o graf -lm

main.o: main.c fruchterman.h graph_utils.h pars.h output.h graph.h
    $(CC) $(CFLAGS) -c main.c

fruchterman.o: fruchterman.c fruchterman.h
    $(CC) $(CFLAGS) -c fruchterman.c

graph_utils.o: graph_utils.c graph_utils.h
    $(CC) $(CFLAGS) -c graph_utils.c

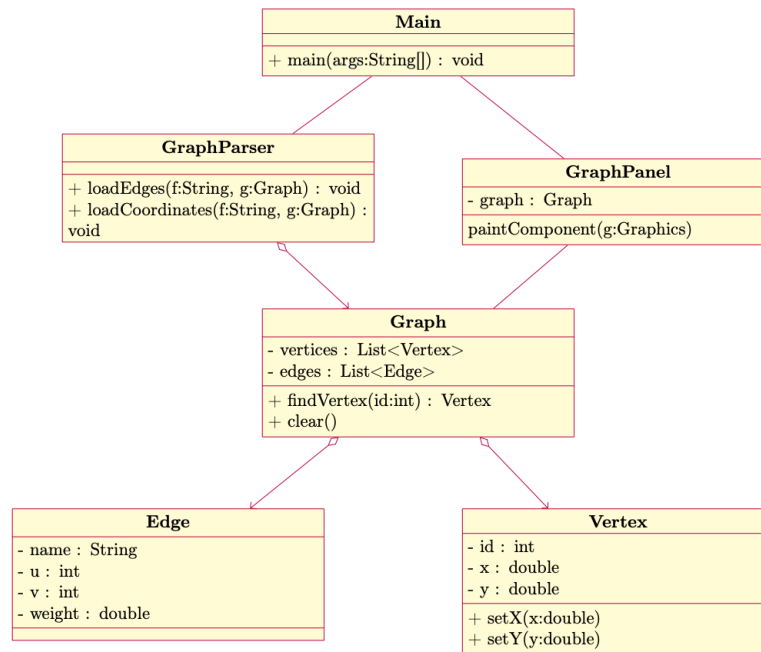
output.o: output.c output.h
    $(CC) $(CFLAGS) -c output.c

pars.o: pars.c pars.h
    $(CC) $(CFLAGS) -c pars.c

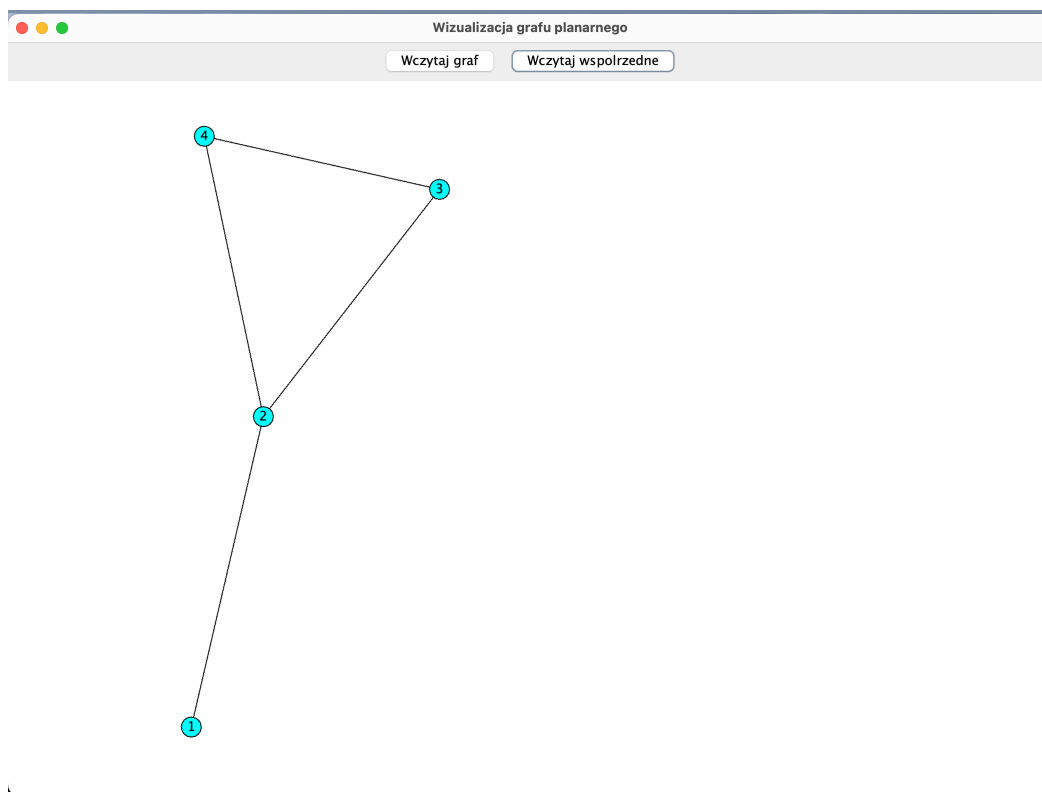
clean:
    rm -f *.o graf out.txt out.bin
```

Część w Javie:

Cały program składa się z sześciu klas.



Graficzny interfejs użytkownika składa się z okna oraz dwóch przycisków do wczytywania plików.



W klasach Vertex oraz Edge przechowywane są informacje o wierzchołku i krawędzi grafu.

```
public class Vertex { 11 usages
    private final int id; 2 usages

    private double x; 2 usages
    private double y; 2 usages

    public Vertex(int id) { 2 usages
        this.id = id;
    }

    public int getId() { 2 usages
        return id;
    }

    public double getX() { 3 usages
        return x;
    }

    public double getY() { 3 usages
        return y;
    }

    public void setX(double x) { this.x = x; }

    public void setY(double y) { this.y = y; }
}
```

```

public class Edge { 5 usages
    private final String name; 2 usages

    private final int u; 2 usages
    private final int v; 2 usages

    private final double weight; 2 usages

    public Edge(String name, int u, int v, double weight) { 1 usage
        this.name = name;
        this.u = u;
        this.v = v;
        this.weight = weight;
    }

    public String getName() { return name; }

    public int getU() { return u; }

    public int getV() { return v; }

    public double getWeight() { return weight; }
}

```

Klasa Graph posiada dwie listy: jedna z wierzchołkami, drugą z krawędziami. W tej klasie znajduje się metoda findVertex(), która przeszukuje listę, aby znaleźć wierzchołek po jego numerze id.

```

public Vertex findVertex(int id) { 5 usages
    for (Vertex v : vertices) {
        if (v.getId() == id)
            return v;
    }

    return null;
}

```


Klasa GraphParser posiada dwie metody, które służą do czytania danych z plików. Znajdują się w niej dwie metody loadEdges() oraz loadCoordinates().

```
public static void loadEdges(String filename, Graph graph) throws Exception { 1 usage

    graph.clear();

    BufferedReader br = new BufferedReader(new FileReader(filename));

    String line;

    while ((line = br.readLine()) != null) {

        String[] parts = line.trim().split(regex: "\\s+");

        if (parts.length < 4)
            continue;

        String name = parts[0];

        int u = Integer.parseInt(parts[1]);
        int v = Integer.parseInt(parts[2]);

        double weight = Double.parseDouble(parts[3]);

        graph.addEdge(new Edge(name, u, v, weight));

        if (graph.findVertex(u) == null)
            graph.addVertex(new Vertex(u));

        if (graph.findVertex(v) == null)
            graph.addVertex(new Vertex(v));
    }

    br.close();
}
```

```

public static void loadCoordinates(String filename, Graph graph) throws Exception { 1 usage

    BufferedReader br = new BufferedReader(new FileReader(filename));

    String line;

    while ((line = br.readLine()) != null) {

        String[] parts = line.trim().split(regex: "\\s+");

        if (parts.length < 3)
            continue;

        int id = Integer.parseInt(parts[0]);

        double x = Double.parseDouble(parts[1]);
        double y = Double.parseDouble(parts[2]);

        Vertex vertex = graph.findVertex(id);

        if (vertex != null) {

            // Skalowanie żeby było lepiej widać
            vertex.setX(x * 5);
            vertex.setY(y * 5);

        }

    }

    br.close();
}

```

Klasa GraphPanel odpowiada za wizualizację grafu. W tym celu została użyta biblioteka Swing.

```
// Rysowanie krawędzi
for (Edge edge : graph.getEdges()) {

    Vertex v1 = graph.findVertex(edge.getU());
    Vertex v2 = graph.findVertex(edge.getV());

    if (v1 == null || v2 == null)
        continue;

    int x1 = (int) v1.getX() + 100;
    int y1 = (int) v1.getY() + 100;

    int x2 = (int) v2.getX() + 100;
    int y2 = (int) v2.getY() + 100;

    g2.setColor(Color.BLACK);
    g2.drawLine(x1, y1, x2, y2);
}
}
```

```
// Rysowanie wierzchołków
for (Vertex vertex : graph.getVertices()) {

    int x = (int) vertex.getX() + 100;
    int y = (int) vertex.getY() + 100;

    g2.setColor(Color.CYAN);
    g2.fillOval(x: x - 10, y: y - 10, width: 20, height: 20);

    g2.setColor(Color.BLACK);
    g2.drawOval(x: x - 10, y: y - 10, width: 20, height: 20);

    g2.drawString(
        String.valueOf(vertex.getId()),
        x: x - 4,
        y: y + 4
    );
}
}
```

Klasa Main odpowiada za utworzenie okna, gdzie jest wyświetlany graf, interaktywnych przycisków oraz uruchomienie programu rysowania grafu.

```
SwingUtilities.invokeLater(() -> {  
  
    JFrame frame = new JFrame( title: "Wizualizacja grafu planarnego");  
  
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    frame.setSize( width: 1000, height: 800);  
  
    GraphPanel panel = new GraphPanel();  
  
    JButton loadGraphButton = new JButton( text: "Wczytaj graf");  
    JButton loadCoordsButton = new JButton( text: "Wczytaj współrzędne");  
  
    JPanel topPanel = new JPanel();  
  
    topPanel.add(loadGraphButton);  
    topPanel.add(loadCoordsButton);  
}
```

```
// Wczytywanie grafu
loadGraphButton.addActionListener( ActionEvent e -> {

    JFileChooser chooser = new JFileChooser();

    int result = chooser.showOpenDialog(frame);

    if (result == JFileChooser.APPROVE_OPTION) {

        File file = chooser.getSelectedFile();

        try {

            GraphParser.loadEdges(
                file.getAbsolutePath(),
                panel.getGraph()
            );

            panel.repaint();

        } catch (Exception ex) {

            JOptionPane.showMessageDialog(frame, message: "Błąd wczytywania grafu");

        }

    }

});
```

```
// wczytywanie współrzędnych
loadCoordsButton.addActionListener( ActionEvent e -> {

    JFileChooser chooser = new JFileChooser();

    int result = chooser.showOpenDialog(frame);

    if (result == JFileChooser.APPROVE_OPTION) {

        File file = chooser.getSelectedFile();

        try {

            GraphParser.loadCoordinates(
                file.getAbsolutePath(),
                panel.getGraph()
            );

            panel.repaint();

        } catch (Exception ex) {

            JOptionPane.showMessageDialog(
                frame,
                message: "Błąd wczytywania współrzędnych"
            );

        }

    }
}
```

7. Komunikaty błędów

W przypadku podania nieprawidłowych danych program wyświetli komunikaty:

- brak podania parametru zapisu pliku: „Proszę podać w jakim formacie mają być zapisane pliki”
- nieprawidłowy parametr: „Błędny parametr”
- ujemna waga lub wierzchołki: „Wprowadzone wartości są błędne”
- niepoprawny format pliku wejściowego: „Nie można odczytać pliku”
- przy podaniu złych plików wyświetli się komunikat: „Błąd wczytywania grafu/współrzędnych”

8. Testy

Testowanie prawidłowo podanych danych:

dane wejściowe:

```
AB 1 2 1
BC 2 3 1
CD 3 4 1
DA 4 1 1.407
```

Uruchomienie:

```
→ Projekt-graf git:(main) x make
cc -Wall -Wextra -c fruchterman.c
cc -Wall -Wextra -c graph_utils.c
cc -Wall -Wextra -c main.c
cc -Wall -Wextra -c output.c
cc -Wall -Wextra -c pars.c
cc fruchterman.o graph_utils.o main.o output.o pars.o -o graf -lm
→ Projekt-graf git:(main) x ./graf graf.txt -t
→ Projekt-graf git:(main) x □
```

Wynik:

```
4 7.633 9.226
1 57.078 55.819
3 47.069 10.353
2 21.762 73.404
```

Testowanie nieprawidłowych danych:

dane wejściowe:

```
AB 1 2 0
BC 2 3 1
CD 3 4 1
DA 4 1 1.407
```

Uruchomienie:

```
● → Projekt-graf git:(main) ✗ make
cc -Wall -Wextra -c fruchterman.c
cc -Wall -Wextra -c graph_utils.c
cc -Wall -Wextra -c main.c
cc -Wall -Wextra -c output.c
cc -Wall -Wextra -c pars.c
cc fruchterman.o graph_utils.o main.o output.o pars.o -o graf -lm
⊗ → Projekt-graf git:(main) ✗ ./graf graf.txt -t
Wprowadzone wartosci sa bledne
```


Testy wizualizacji:

Dane wejściowe:

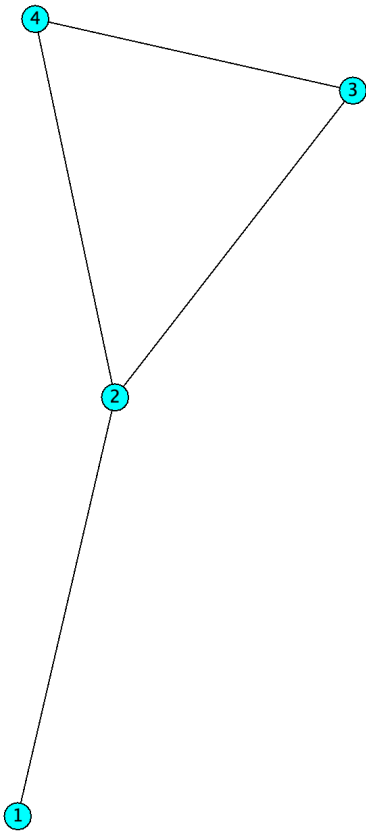
AB	1	2	1
BC	2	3	1
CD	3	4	1
DA	4	1	1.407

rys a graf

4	19.285	-9.111
2	31.133	47.189
3	66.326	1.637
1	16.799	109.086

rys b współrzędne

Wynik działania:



Dane wejściowe:

A	1	2	10
B	2	3	12
C	3	1	15

rys c graf

3	9.659	28.723
1	61.107	54.923
2	58.073	-2.732

rys d współrzędne

Wynik działania:

